

Eliminación de Variables en Diagramas de Influencia

Arquitectura e implementación en OpenMarkov

Documentación técnica — OpenMarkov 0.3.0-SNAPSHOT

Abril 2026

Índice

1. Introducción	2
2. Fundamentos matemáticos	2
2.1. Eliminación de una variable de azar	2
2.2. Eliminación de una variable de decisión	2
3. Clases principales	2
3.1. VariableEliminationCore	3
3.2. ChanceVariableElimination	3
3.3. DecisionVariableElimination	3
3.4. VariableElimination (abstracta)	4
3.5. VEEvaluation	4
4. Clases de soporte	4
5. Tareas VE para diagramas de influencia	5
6. Heurísticas de eliminación	5
7. Flujo completo del algoritmo	6
8. Ejemplo de uso	6
9. Diagrama UML	7
9.1. Diagrama de clases	7
9.2. Diagrama de actividad	7
9.3. Diagrama de secuencia	7

1. Introducción

La **eliminación de variables** (VE) es el motor principal de inferencia exacta en OpenMarkov. Mientras que en redes bayesianas cada variable se elimina por *marginalización* (suma), en un **diagrama de influencia** (ID) existen dos operaciones distintas según el tipo de nodo:

- **Nodo de azar (chance):** se elimina por marginalización — $\sum_x P(x \mid \text{pa}(x)) \cdot U(x, \dots)$.
- **Nodo de decisión:** se elimina por maximización — $\max_d U(d, \dots)$, obteniendo además la *política óptima* $\delta^*(d)$.

El orden de eliminación respeta el **orden parcial de información** del diagrama, y se determina mediante una heurística configurable.

2. Fundamentos matemáticos

Sea un diagrama de influencia con variables de azar $C = \{C_1, \dots, C_m\}$, variables de decisión $D = \{D_1, \dots, D_k\}$ y funciones de utilidad $U_1, \dots, U_r$. La utilidad esperada máxima es:

$$\text{MEU} = \max_{D_k} \sum_{C_{i_k}} \cdots \max_{D_1} \sum_{C_{i_1}} \left(\prod_j P(C_j \mid \text{pa}(C_j)) \right) \left(\sum_l U_l \right) \quad (1)$$

donde el orden de las operaciones $\max$ y $\sum$ viene dado por el orden parcial de información.

2.1. Eliminación de una variable de azar

Dada una variable de azar X con potenciales de probabilidad $\{\phi_1, \dots, \phi_p\}$ y potenciales de utilidad $\{U_1, \dots, U_q\}$ que dependen de X :

1. Calcular la probabilidad conjunta: $\phi_{\text{joint}} = \prod_{i=1}^p \phi_i$
2. Marginalizar: $\phi_{\text{marg}} = \sum_X \phi_{\text{joint}}$
3. Calcular la probabilidad condicional: $P(X \mid \text{resto}) = \phi_{\text{joint}} / \phi_{\text{marg}}$
4. Para cada potencial de utilidad U_k , calcular la utilidad esperada: $U'_k = \sum_X P(X \mid \text{resto}) \cdot U_k$

Los nuevos potenciales ϕ_{marg} y $\{U'_1, \dots, U'_q\}$ reemplazan a los originales en la red.

2.2. Eliminación de una variable de decisión

Dada una variable de decisión D con potenciales de probabilidad $\{\phi_1, \dots, \phi_p\}$ y utilidad total U_{total} :

1. Calcular la probabilidad conjunta: $\phi_{\text{joint}} = \prod_{i=1}^p \phi_i$
2. Proyectar la probabilidad: $\phi_{\text{proj}} = \text{projectOut}(D, \phi_{\text{joint}})$
3. Maximizar la utilidad: $U^* = \max_D U_{\text{total}}$
4. Obtener la política óptima: $\delta^*(D) = \arg \max_D U_{\text{total}}$

3. Clases principales

Todas las clases residen en el módulo `org.openmarkov.inference`, paquete `algorithm.variableElimination`

3.1. VariableEliminationCore

algorithm/variableElimination/VariableEliminationCore.java

Motor central del algoritmo. Recibe una *red de decisión de Markov* (ya preprocesada), una heurística de eliminación y un indicador de unicriterio/bicriterio.

- `performVariableElimination()`: bucle principal — pide a la heurística la siguiente variable y llama a `eliminateVariable()`.
- `eliminateVariable()`: según `NodeType`:
 - CHANCE → crea un `ChanceVariableElimination`
 - DECISION → crea un `DecisionVariableElimination`
- Mantiene un `LinkedHashMap<Variable, TablePotential>` con las políticas óptimas (en orden de inserción = orden de eliminación).
- `getUtility()`: devuelve la utilidad global esperada tras la eliminación.
- `getProbability()`: devuelve la probabilidad conjunta residual.

3.2. ChanceVariableElimination

algorithm/variableElimination/ChanceVariableElimination.java

Elimina una variable de azar por marginalización:

1. Multiplica los potenciales de probabilidad → `jointProbability`.
2. Marginaliza sobre la variable → `marginalProbability`.
3. Divide para obtener la condicional → `conditionalProbability`.
4. Para cada potencial de utilidad, calcula `multiplyAndMarginalize(condicional, utilidad, variable)`.

Soporta tanto `TablePotential` (unicriterio) como `GTablePotential` (bicriterio/coste-efectividad) vía `CEAlgebra`.

3.3. DecisionVariableElimination

algorithm/variableElimination/DecisionVariableElimination.java

Elimina una variable de decisión por maximización:

1. Suma los potenciales de utilidad → `totalUtility`.
2. Multiplica y proyecta los potenciales de probabilidad.
3. Maximiza:
 - Unicriterio: usa `MaxOutVariable` → obtiene utilidad máxima y política óptima.
 - Bicriterio: usa `CEAlgebra.ceMaximize` → maximización sobre particiones coste-efectividad.

3.4. VariableElimination (abstracta)

algorithm/variableElimination/tasks/VariableElimination.java

Clase base abstracta para todas las tareas VE. Extiende `InferenceAlgorithm` (del módulo core). Proporciona:

- `generalPreprocessing()`: expansión temporal, imposición de políticas, extensión de evidencia pre-resolución.
- `exactAlgorithmsPreprocessing()`: discretización de variables numéricas, absorción de nodos numéricos intermedios.
- `unicriterionPreprocess()` / `bicriteriaPreprocess()`: escalado de utilidades.
- `heuristicFactory()`: crea la heurística de eliminación proyectando el orden parcial.

3.5. VEEvaluation

algorithm/variableElimination/tasks/VEEvaluation.java

Tarea concreta para evaluar un diagrama de influencia. Implementa las interfaces `Evaluation` y `OptimalPolicies`. Su método `resolve()`:

1. Ejecuta el preprocesamiento general, unicriterio y exacto.
2. Construye la red de Markov-decisión proyectada via `TaskUtilities`.
3. Determina las variables a eliminar (chance + decisión).
4. Crea la heurística y lanza `VariableEliminationCore`.
5. Expone: `getProbability()`, `getUtility()`, `getOptimalPolicies()`, `getOptimalStrategyTree()`.

4. Clases de soporte

Clase	Responsabilidad
<code>EliminationHeuristic</code> (org.openmarkov.core)	Interfaz/clase abstracta que determina el orden de eliminación. Implementaciones: <code>SimpleElimination</code> , <code>MinimalFillIn</code> , <code>CanoMoralElimination</code> , <code>WeightedMinFill</code> , etc.
<code>DiscretePotentialOperations</code> (org.openmarkov.core)	Operaciones sobre <code>TablePotential</code> : <code>multiply</code> , <code>marginalize</code> , <code>multiplyAndMarginalize</code> , <code>divide</code> , <code>sum</code> , <code>projectOutVariable</code> , <code>sumByCriterion</code> .
<code>MaxOutVariable</code> (org.openmarkov.core)	Maximización de una variable de decisión: devuelve la utilidad máxima y la política óptima (arg máx).
<code>VariableEliminationOperations</code>	Ordena potenciales de utilidad según el orden parcial de decisiones (interventions en árboles de estrategia).
<code>CEAlgebra</code>	Álgebra para análisis coste-efectividad: <code>ceMaximize</code> y <code>multiplyAndMarginalize</code> sobre <code>GTablePotential</code> .

Clase	Responsabilidad
CreatePotentialUtility	Crea potenciales CEP (particiones coste-efectividad) a partir de potenciales de coste y efectividad.
TaskUtilities (org.openmarkov.core)	Preprocesamiento: construye la red de Markov-decisión, proyecta tablas, extiende evidencia, impone políticas, discretiza variables.
BasicOperations (org.openmarkov.core)	projectPartialOrder: proyecta el orden parcial de variables sobre el subconjunto a eliminar.

5. Tareas VE para diagramas de influencia

Además de VEEvaluation, existen otras tareas VE aplicables a IDs:

Clase	Interfaz	Descripción
VEPropagation	Propagation	Marginales posteriores sobre BN/ID
VEEvaluation	Evaluation, OptimalPolicies	Evaluación completa: $P(E)$, MEU, políticas óptimas
VEExpectedUtility- Decision	ExpectedUtility- Decision	Utilidad esperada desglosada por decisión
VEOptimalPolicies	OptimalPolicies	Políticas óptimas de decisión
VEOptimalIntervention	OptimalIntervention	Intervención óptima
VECEAnalysis	CEAnalysis	Análisis coste-efectividad (bicriterio)
VECEPSA	CE_PSA	Análisis de sensibilidad probabilística para CE

6. Heurísticas de eliminación

El orden de eliminación afecta drásticamente al tamaño de los potenciales intermedios. Las heurísticas disponibles en `org.openmarkov.inference.heuristic`:

Heurística	Criterio de selección
SimpleElimination	Orden natural (por defecto) — sigue el orden parcial sin optimizar
MinimalFillIn	Minimiza el número de aristas añadidas al grafo moral
minimalCliqueSize	Minimiza el tamaño del clique resultante
CanoMoralElimination	Heurística de Cano y Moral para diagramas de influencia
WeightedMinFill	Min-fill ponderado por tamaño de tabla
LookaheadMinFill / LookaheadMinCliqueSize	Versión con lookahead (mira un paso adelante)
HybridElimination	Combinación de varias heurísticas
TimeSliceElimination	Especializada para redes bayesianas dinámicas (DBN)

Heurística	Criterio de selección
FileElimination	Orden leído desde un fichero externo

7. Flujo completo del algoritmo

El diagrama PlantUML adjunto (`ve-influence-diagrams.puml`) muestra el flujo completo. A continuación se describe textualmente:

1. **Entrada:** red probabilística (ID), evidencia pre-resolución, políticas impuestas (opcionales).
2. **Preprocesamiento general** (`VariableElimination`):
 - a) Si es temporal, expandir la red.
 - b) Convertir decisiones con políticas impuestas en nodos de azar.
 - c) Extender la evidencia pre-resolución.
 - d) Si es temporal, aplicar descuentos.
3. **Preprocesamiento exacto:**
 - a) Discretizar variables numéricas no observadas.
 - b) Absorber nodos numéricos intermedios.
4. **Construcción de la red de Markov-decisión:** `TaskUtilities.projectTablesAndBuildMarkovDecisionProcess`
5. **Creación de la heurística:** se proyecta el orden parcial de variables y se instancia la heurística configurada.
6. **Bucle de eliminación** (`VariableEliminationCore`):
 - a) Pedir a la heurística la siguiente variable X .
 - b) Si X es `null`, terminar.
 - c) Extraer los potenciales de probabilidad y utilidad que dependen de X .
 - d) Eliminar el nodo de la red.
 - e) Si X es de azar: crear `ChanceVariableElimination` $\rightarrow$ marginalizar.
 - f) Si X es de decisión: crear `DecisionVariableElimination` $\rightarrow$ maximizar y guardar política óptima.
 - g) Añadir los nuevos potenciales a la red.
 - h) Volver al paso (a).
7. **Salida:** utilidad esperada máxima (MEU), probabilidad de la evidencia, políticas óptimas para cada decisión.

8. Ejemplo de uso

```
// Cargar la red
ProbNet probNet = loadNetwork("ID-arthronet.pgm");

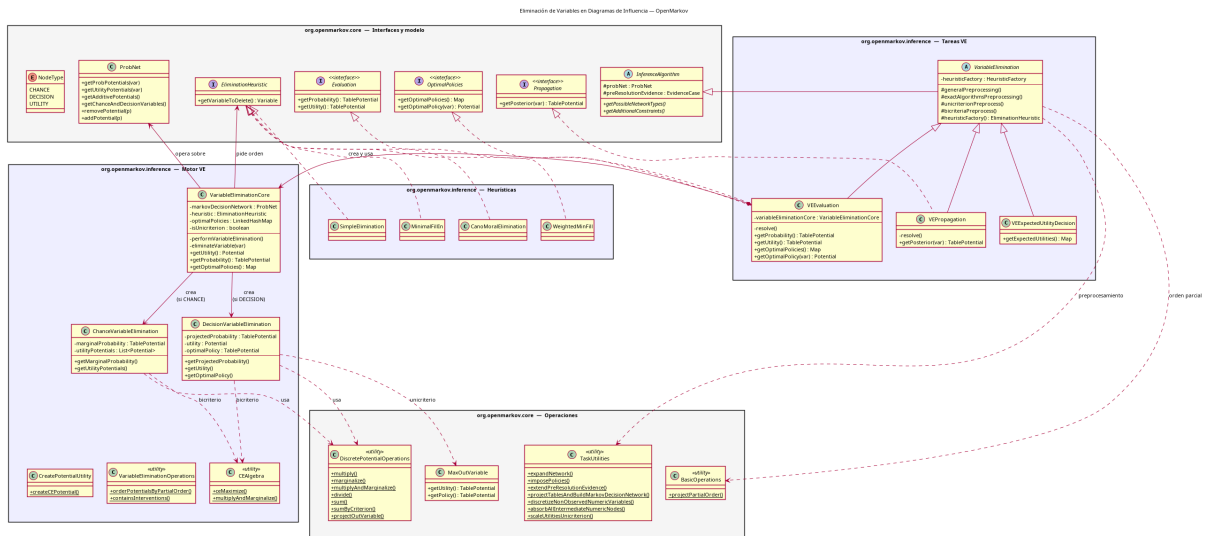
// Crear el algoritmo
VEEvaluation evaluation = new VEEvaluation(probNet);
evaluation.setPreResolutionEvidence(evidence);

// Obtener resultados
TablePotential utility = evaluation.getUtility();
TablePotential probability = evaluation.getProbability();
HashMap<Variable, Potential> policies =
    evaluation.getOptimalPolicies();

// Acceder a la politica de una decision concreta
Potential policy = evaluation.getOptimalPolicy(decisionVar);
```

Los diagramas siguientes se generan a partir del fichero `ve-influence-diagrams.puml` mediante PlantUML (`make uml`).

Muestra la jerarquía de clases VE, las dependencias con el core y las interfaces de tarea.



9.2. Diagrama de actividad

9.3. Diagrama de secuencia

7

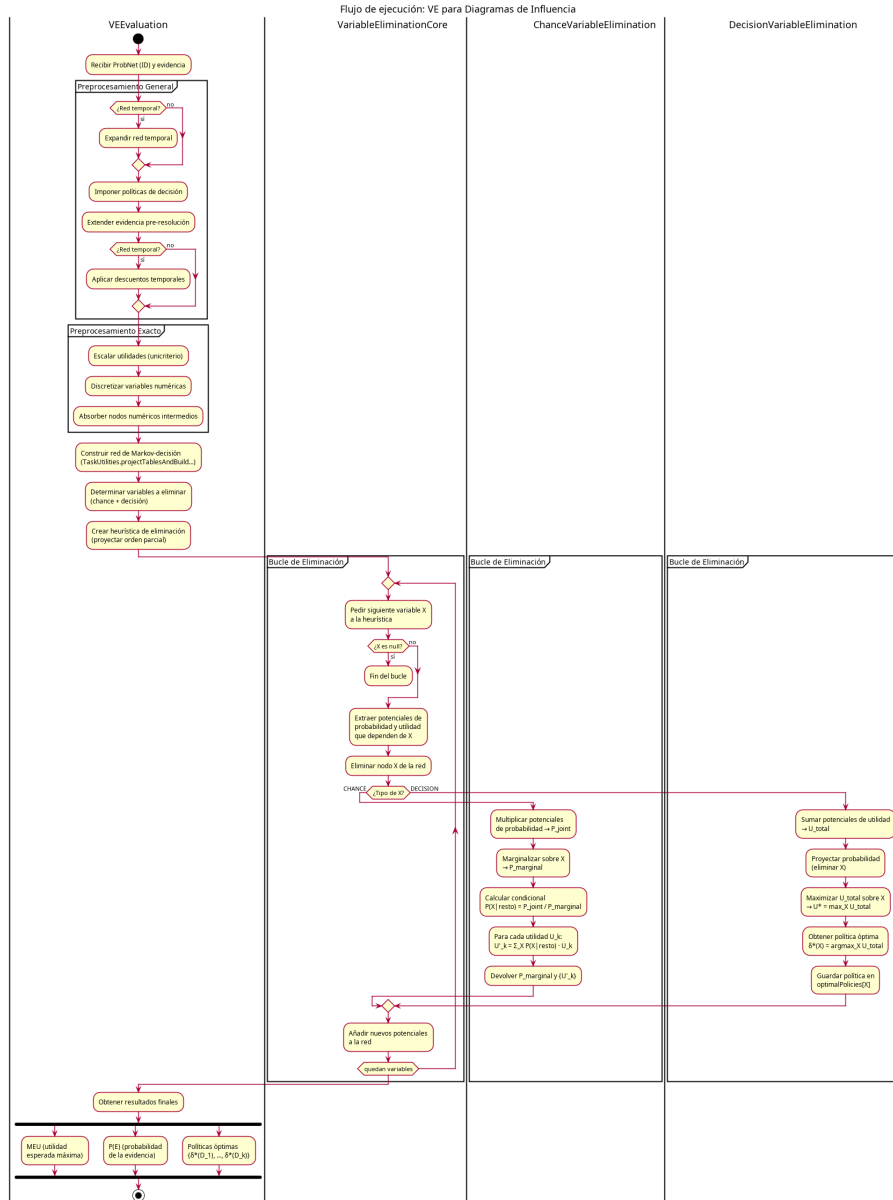


Figura 2: Diagrama de actividad del algoritmo VE para IDs.

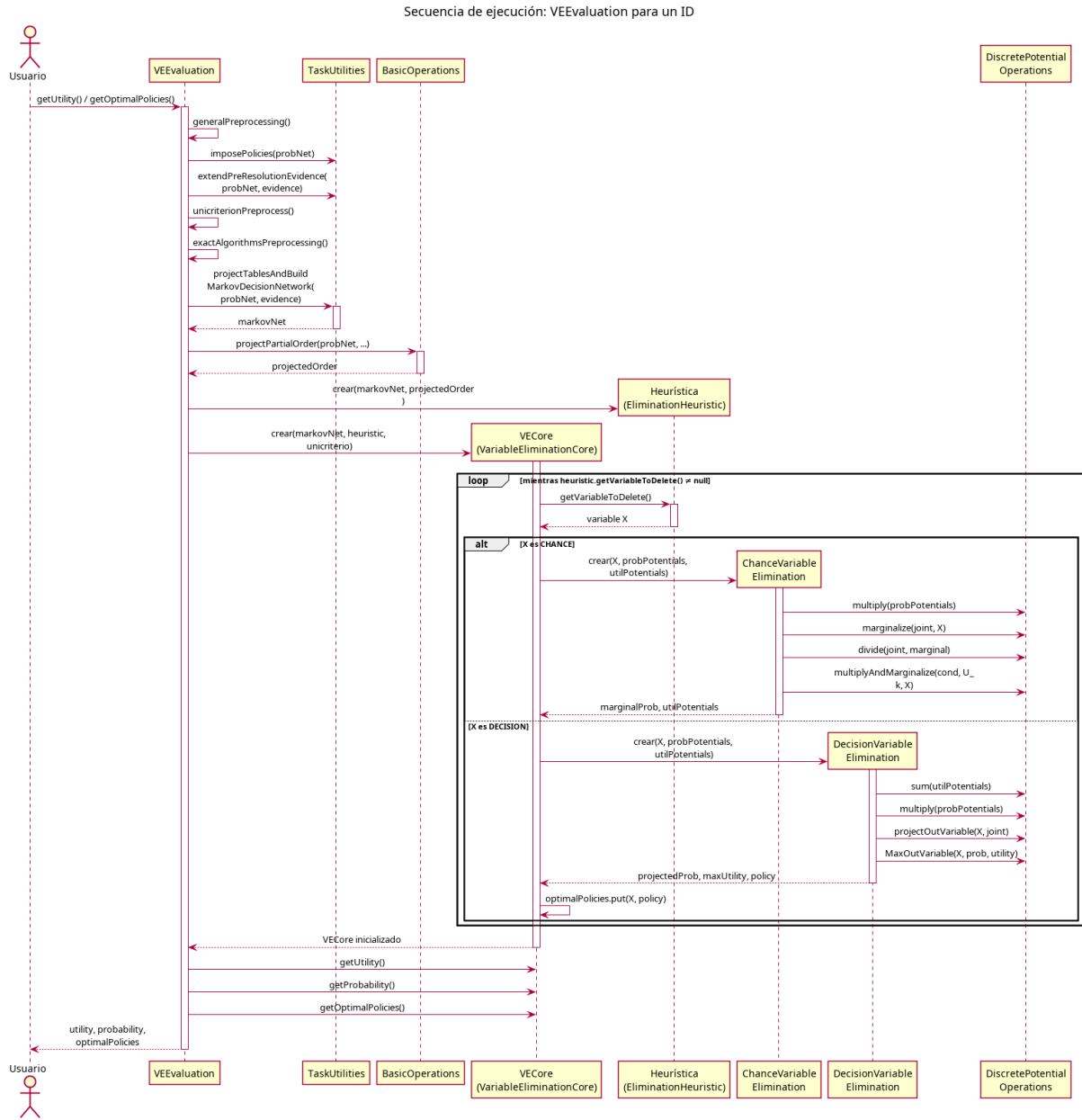


Figura 3: Diagrama de secuencia de VEEvaluation.